MANIFESTE D'ARCHITECTURE TECHNIQUE & LIVRE BLANC SYSTÈME

Ce document formalise les fondements d'ingénierie, les choix de conception matérielle et les contraintes opérationnelles régissant le projet **RAP-EDIT**, un éditeur de texte et d'inspection binaire conçu pour les environnements de crise, les terminaux de secours et les infrastructures critiques sans bibliothèque dynamique.

1. Philosophie et doctrine opérationnelle

La plupart des éditeurs modernes (Vim, Nano, Emacs, Micro) reposent sur une hypothèse implicite : le système d'exploitation hôte est sain, la mémoire vive est abondante et les bibliothèques partagées dynamiques (`libc.so`) sont intactes. Dans un contexte de sauvetage d'urgence (système corrompu, racine montée en lecture seule, coupure de courant brutale, corruption de glibc ou mémoire saturée), ces hypothèses s'effondrent :

- Une allocation dynamique `malloc()` peut déclencher le **Linux OOM Killer** (*Out-Of-Memory Killer*) qui élimine arbitrairement le processus de l'éditeur.
- La perte d'une bibliothèque partagée empêche le lancement du binaire (`error while loading shared libraries`).
- L'absence d'appel explicite à `fsync()` lors de la sauvegarde laisse les données dans le cache volatile du noyau, les détruisant si la machine redémarre brutalement.

RAP-EDIT est pensé pour garantir une disponibilité totale sous contrainte maximale : il doit pouvoir démarrer, auditer, assainir, réparer et persister physiquement des fichiers critiques même lorsque le reste du système est en échec partiel.

2. Modèle mémoire déterministe (Doctrine Zero-Heap / BSS)

L'interdiction absolue d'allouer de la mémoire sur le tas (*heap*) est la règle d'or de RAP-EDIT.

```
Textproto
+-------------------------------------------------------------------------+
| MÉMOIRE VIRTUELLE DU PROCESSUS RAP-EDIT                                 |
+-------------------------------------------------------------------------+
| [TEXT] Instructions machine immuables (Code binaire ELF statique)      |
+-------------------------------------------------------------------------+
| [DATA] Variables globales initialisées                                  |
+-------------------------------------------------------------------------+
| [BSS]  TAMPONS STATIQUES PRÉ-ALLOUÉS AU DÉMARRAGE (~2.7 Mo fixes)       |
|        - Tampons de travail : 4 onglets x 256 Ko (raw_buffer)           |
|        - Indexation de lignes : 4 onglets x 8192 entrées (rap_line_t)   |
|        - Tampon d'annulation : Instantané miroir de secours             |
|        - Presse-papiers inter-onglets partagé (256 Ko)                  |
|        - Tampon miroir de diff : mod_diff (256 Ko + 16384 deltas)       |
+-------------------------------------------------------------------------+
| [HEAP] ABSENT (malloc / calloc / realloc STRICTEMENT BANNIS)            |
+-------------------------------------------------------------------------+
| [STACK] Pile d'exécution locale (variables éphémères bornées)           |
+-------------------------------------------------------------------------+
```

Principes fondamentaux du modèle BSS :

1. **Empreinte mémoire prévisible :** L'éditeur occupe une place fixe et constante en RAM dès la première microseconde de son exécution. Aucun pic de mémoire ne peut survenir en cours d'utilisation, quelle que soit la commande tapée.
2. **Immunité contre le Linux OOM Killer :** Ne demandant jamais de pages mémoire supplémentaires au noyau via brk() ou mmap(), RAP-EDIT ne présente aucun risque de provoquer un *Out-Of-Memory* et ne figure pas parmi les cibles prioritaires d'élimination par le noyau.
3. **Zéro fragmentation :** L'absence de cycles d'allocations/désallocations élimine totalement la fragmentation de l'espace d'adressage virtuel, garantissant un comportement déterministe après des jours d'exécution continue.
4. **Calcul d'offset en $O(1)$:** L'indexation matricielle fixe permet des opérations arithmétiques directes pour le déplacement, le défilement et la projection hexadécimale.

3. Autonomie binaire et liaison statique pure

RAP-EDIT est compilé avec l'indicateur -static, produisant un exécutable autonome auto-suffisant.

Indépendance vis-à-vis du chargeur dynamique :

```
Textproto
$ file bin/rap-edit
bin/rap-edit: ELF 64-bit LSB executable, x86-64, statically linked, stripped
$ ldd bin/rap-edit
not a dynamic executable
```

- **Bénéfice tactique :** Le binaire peut être copié sur une clé USB de secours, un initramfs minimal, une partition de maintenance BusyBox ou un conteneur dégradé, et s'exécutera sans nécessiter aucune dépendance externe, paquet tiers ou bibliothèque système.

4. Persistance physique matérielle (Barrière synchrone `fsync`)

L'appel système standard `write()` sous Linux se contente de transférer les octets vers le *Page Cache* du noyau. Si une coupure d'alimentation survient quelques secondes après, les données sont perdues et le fichier cible se retrouve fréquemment tronqué ou rempli d'octets nuls (`0x00`).

RAP-EDIT intègre un protocole d'écriture sécurisé à trois niveaux :

```
Textproto
[TAMPON RAM] (raw_buffer)
     |
     ▼ (open O_WRONLY | O_CREAT | O_TRUNC, 0644)
[CACHE NOYAU LINUX] (Page Cache)
     |
     ▼ fsync(fd)  <--- BARRIÈRE MATÉRIELLE OBLIGATOIRE
[DISQUE PHYSIQUE / FLASH SSD] (Blocs magnétiques / Cellules NAND confirmées)
     |
     ▼ (close)
[STATUT DU COCKPIT] -> [SYNCHRONISÉ]
```

1. **Sauvegarde normale (`:w!`) :** Écriture intégrale suivie immédiatement d'un `fsync()` bloquant pour forcer le contrôleur de stockage à vider ses caches internes avant de rendre la main.
2. **Clonage physique de référence (`:bak orig`) :** Lecture bloc par bloc du fichier déjà présent sur disque vers `<fichier>.orig.bak` sans passer par la mémoire d'édition, figeant l'état d'origine du système.
3. **Évacuation de panique (`:panic`) :** En cas d'impossibilité d'écrire dans le répertoire courant (ex: disque saturé ou partition verrouillée en lecture seule), le moteur redirige automatiquement l'évacuation physique vers `/tmp/` ou un dossier de secours avec écriture synchrone garantie.

5. Architecture modulaire à greffons BSS (`rap_plugin_t`)

Pour respecter les standards de modularité stricte, le noyau central de RAP-EDIT (`rap_edit_core.c` et `rap_edit_events.c`) ne contient aucun code métier dédié à une tâche spécifique. Chaque fonctionnalité est encapsulée dans un greffon autonome dans `src/plugins/`.

```C/C++
typedef int (*rap_plugin_fn_t)(rap_context_t *ctx, const char *args);

typedef struct {
    char            cmd_trigger[16];   /* Mot-clé déclencheur (ex: "diff",
"hex", "ro") */
    char            description[64];   /* Rôle documenté du greffon */
    rap_plugin_fn_t execute;           /* Pointeur de fonction vers la routine
BSS */
} rap_plugin_t;
```

Répertoire des greffons intégrés :

Greffon	Rôle tactique	Déclencheurs CLI
mod_buffer	Gestion multi-onglets (jusqu'à 4 tampons simultanés en BSS)	:b, :bd, :close, :1..:4
mod_text	Insertion, suppression unitaire, retour à la ligne et UTF-8	Frappe directe, touches clavier
mod_io	Chargement synchrone et écriture matérielle avec fsync()	:w!, lecture initiale
mod_clipboard	Presse-papiers unifié partagé inter-onglets et macros directes	:1:10c, :15w, :10x:15w, Ctrl+C/V
mod_nav	Navigation rapide, coordonnées de fin de fichier et bornes	:$, :<num>, Ctrl+A/E, PgUp/PgDn
mod_search	Recherche circulaire instantanée sans regex lourde	:rch <motif>
mod_undo	Tampon miroir d'annulation et de rétablissement instantané	:u, Ctrl+Z, Ctrl+U
mod_delete	Suppression courante, par lot relatif ou par plage absolue	:d, :dd, :<N>dd, :<num>d, :<deb>:<fin>d
mod_replace	Moteur de substitution de chaînes 0-heap	:s/.../, :%s/.../g, :<d>:<f>s/.../g

mod_sanitize	Détection et purge chirurgicale des corruptions (\r, 0x00)	:clean, :dos2unix, :strip-null
mod_panic	Sauvegarde atomique synchrone et évacuation de crise	:bak, :bak <cible>, :bak orig, :panic
mod_diff	Comparaison in-situ (RAM vs Clone disque ou inter-onglets)	:diff, :diff orig, :diff #n
mod_hex	Inspecteur hexadécimal brut adaptatif (16/32 octets/ligne)	:hex, :hex <offset>
mod_protect	Verrouillage manuel en lecture seule et protection d'audit	:ro, :rw

6. Matrice des spécifications techniques

Paramètre système	**Spécification RAP-EDIT**	**Justification technique**
Politique d'allocation	**0-Heap (Zéro allocation dynamique)**	Bannissement total de malloc/free pour éviter l'OOM Killer.
Capacité par tampon	**256 Ko (262 144 octets)**	Dimensionné pour la quasi-totalité des fichiers système (/etc).
Nombre max d'onglets	**4 documents simultanés**	Visibilité instantanée sur HUD Cockpit sans surcharge visuelle.
Nombre max de lignes	**8 192 lignes indexées par onglet**	Tableau statique rap_line_t pré-alloué en section .bss.

Presse-papiers unifié	**256 Ko (partagé inter-onglets)**	Permet de copier d'un fichier corrompu pour coller dans un fichier sain.
Persistance disque	**Synchrone avec barrière fsync()**	Élimine le risque de perte de données en cas de crash machine.
Dépendance système	**Autonome (-static, libc liée)**	Exécutable sur tout noyau Linux x86_64 sans bibliothèque partagée.
Protocoles terminaux	**VT100 / VT220 / ANSI purs**	Compatible console physique /dev/tty1, liaisons série et xterm.
Inspection binaire	**Matrice 16/32 octets adaptative**	Analyse géométrique directe sans parsing de saut de ligne (\n).

7. Protocole standardisé d'intervention en environnement dégradé

Textproto

```
Lors d'un incident de production ou d'une panne d'amorçage :
Étape 1 : CHARGEMENT PROTÉGÉ
   ./bin/rap-edit /etc/fichier_critique.conf
   -> Taper immédiatement :ro pour empêcher toute frappe accidentelle pendant
l'état des lieux.

Étape 2 : CLONAGE PHYSIQUE DE SÉCURITÉ
   -> Taper :bak orig
   -> Un clone physique exact est immédiatement écrit et scellé sur disque
(.orig.bak).

Étape 3 : AUDIT DES ANOMALIES
   -> Vérifier le voyant HUD : présence de [CRLF/DOS] ou [!NUL] ?
   -> Taper :hex pour vérifier l'intégrité des octets d'en-tête, BOM UTF-8 ou
caractères non-ASCII.
   -> Quitter avec q.

Étape 4 : ASSAINISSEMENT ET RÉPARATION
   -> Taper :rw pour autoriser les modifications.
   -> Si corrompu : taper :clean pour purger retours chariot et octets nuls en
une passe.
   -> Effectuer les modifications nécessaires dans le texte.

Étape 5 : CONTRÔLE DIFFÉRENTIEL AVANT VALIDATION
   -> Taper :diff orig
   -> Vérifier ligne par ligne les suppressions (-) en rouge et les ajouts (+)
en vert.
   -> Quitter avec q.

Étape 6 : ENGAGEMENT SYNCHRONE SUR DISQUE
   -> Taper :w! pour sceller physiquement les modifications sur le stockage
(fsync).
   -> Quitter proprement avec :q!
```